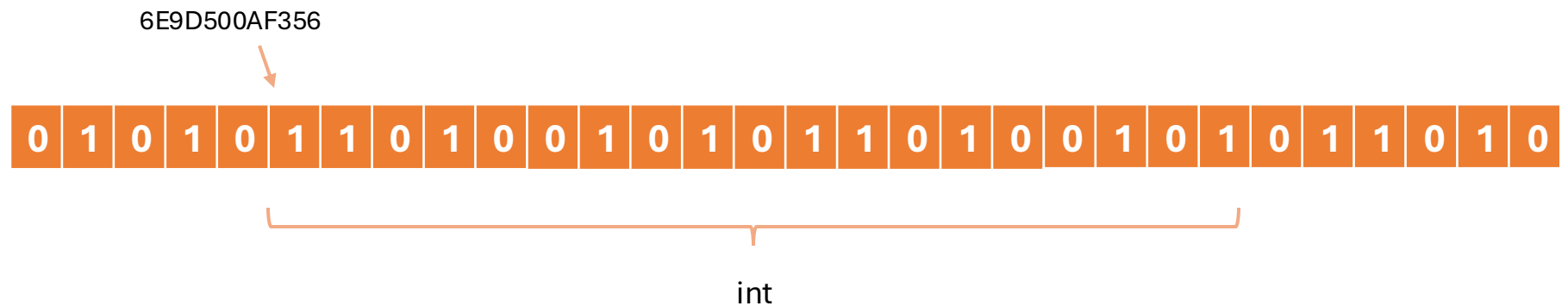


Rappel

Pour accéder et décoder l'information dans un espace mémoire nous avons besoin de :

- Une adresse
 - Première case de l'espace mémoire.
- Un type
 - Taille de l'espace mémoire,
 - Comment interpréter les 0 et 1.



La limite des pointeurs jusque-là...

1. Passer par une variable

2. La durée de vie d'un espace mémoire est soumis à la portée des variables

- Une variable créée dans un bloc est détruite en sortant
- Un espace mémoire alloué dans un bloc est désalloué en sortant

```
int a = 3;  
int * pA = &a;
```

```
int *pA;  
if( ... )  
{  
    int a = 3;  
    *pA = &a;  
}
```

```
printf("%d\n", a);
```

```
printf("%d\n", *pA);
```

Erreur : la variable *a* n'existe plus

Erreur : l'espace mémoire qui avait été créé avec *a* n'existe plus non plus donc *pA* est un pointeur invalide.

La pile : rappel

- Organisation **LIFO** (*Last In, First Out*)
- Type de mémoire **rapide**.
- À chaque nouveau bloc → une **nouvelle zone** ("frame") est empilée
- À la fin d'un bloc → la zone est **dépilée automatiquement**

main : w	3
main() if z v	3
a() : x	1
b() : y	2

```
void c(){  
    int z = 3;  
}
```

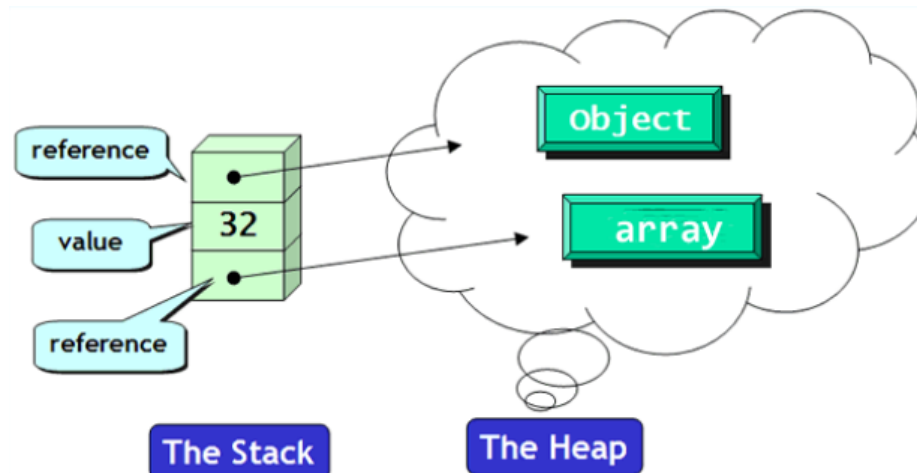
```
void b(){  
    int y = 2;  
}
```

```
void a() {  
    int x = 1;  
    b();  
}
```

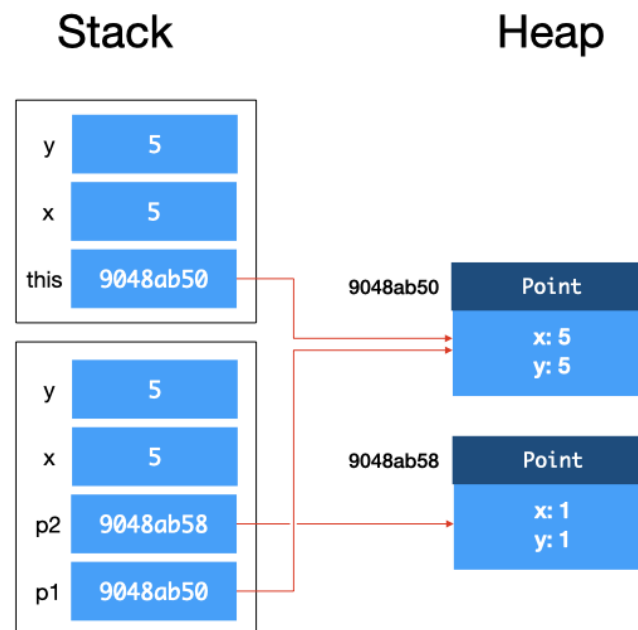
```
int main(){  
    int w = 3;  
    if (w == 3)  
    {  
        int v = 5;  
        a();  
    }  
    c();  
}
```

La mémoire : le tas (heap)

- Zone mémoire dynamique **globale**.
- Conserve des données dont la taille ou la durée de vie ne sont pas connues à l'avance.
- Accès plus lent car les zones ne sont pas les unes à la suite des autres.



La pile et le tas



Allouer de la mémoire oui, mais combien ?

Grace à la fonction :

```
int sizeof(type);
```

```
struct Etudiant
{
    int a;
    bool b;
    bool c;
    double d;
};

int main()
{
    printf("%d\n", sizeof(int));           4
    printf("%d\n", sizeof(bool));         4
    printf("%d\n", sizeof(double));       8
    printf("%d\n", sizeof(Etudiant));     20
}
```

Malloc (en c)

- Crée un espace mémoire d'une certaine taille.
 - Retourne l'adresse de l'espace créé.

Stocker l'adresse renvoyée !

Sinon impossible de retrouver l'espace mémoire.

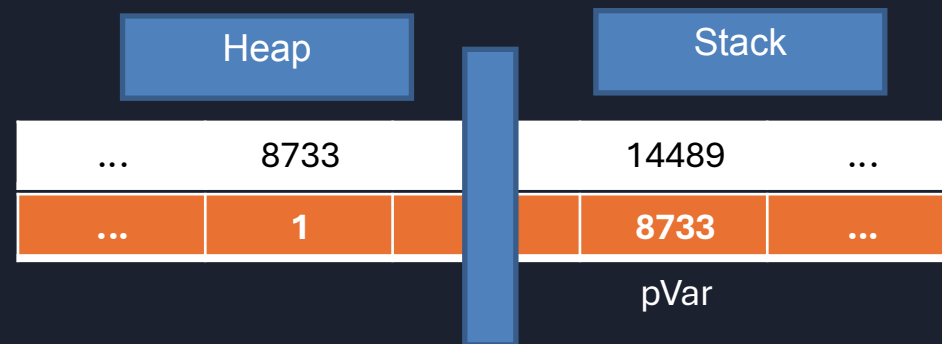
On stocke dans un pointeur

```
#include <stdlib.h>
```

```
void* malloc( taille );
```

```
void* malloc(sizeof(type));
```

```
bool *pVar = malloc(sizeof(bool));  
*pVar = true;
```



```
int *pA;  
if( ... )  
{  
    int a = 3;  
    *pA = &a;  
}
```

```
printf("%d\n", a);
```

```
printf("%d\n", *pA);
```

← Erreur : l'espace mémoire qui avait été créé avec *a* n'existe plus non plus donc *pA* est un pointeur invalide.

Avantages:

- Un espace créé dans le tas avec malloc n'est pas détruit en sortie de bloc.
 - Libérés de la portée des variables.

```
int *pA;  
if ( ... )  
{  
    pA = malloc(sizeof(int));  
    *pA = 3;  
}
```

```
printf("%d\n", *pA);
```

← Crée un nouvel espace mémoire

← Affiche 3 !

Nous pouvons désormais allouer un espace
manuellement.

Mais il faut désormais le désallouer nous même !



WITH GREAT
POWER
COMES GREAT
RESPONSIBILITY

Danger : fuite de mémoire

Accumulation des espaces mémoires qui ne sont pas désalloués manuellement.

Sources du problème :

- Oubli de désallouer sur le tas
- Perte de l'adresse
 - Adresse pas stockée après malloc
 - Dernier pointeur détruit avant désallocation.

Exemple d'erreur :
le pointeur lui est toujours soumis à la portée

```
{  
    int *pA;  
    pA = malloc(sizeof(int));  
    *pA = 3;  
}
```

```
printf("%d\n", *pA);
```

Erreur : pA n'existe plus,
Mais l'espace mémoire qui
était pointé lui existe encore.

...	8733	...	14489	...
...	3	...	8733	...

pA

...	8733	...	...	...
...	3	...	...	...

Free (en c)

Il faut libérer (désallouer) les espaces mémoires alloués dynamiquement.

- Désalloue (ou libère) l'espace mémoire.
- Il ne met pas le pointeur à NULL.

```
int *pAge;  
*pAge = malloc(sizeof(int));  
*pAge = 3;
```



```
free (pAge);  
pAge = NULL;
```

...	8733	...	14489	...
...	3	...	8733	...

pAge

...	...	...	14489	...
...	...	...	8733	...

pAge

Tableau alloué dynamiquement

Définir la taille d'un tableau pendant l'exécution !

(choisir sa taille en fonction de l'utilisation ou des données)

Libérer un tableau comme un pointeur

```
int * arr = malloc( 8 *sizeof(int));  
arr[5] = 3;
```

```
int n;  
scanf("%d", n);  
int * arr = malloc( n *sizeof(int));
```

```
free (arr);
```

Malloc et structure

Tamagotchi.h

```
struct Tamagotchi
{
    int sante;
    double *stats; //tableau de stats
};
```

Main.c

```
int main()
{
    //Allocation dynamique de la structure
    Tamagotchi *t = malloc(sizeof(Tamagotchi));

    //Initialisation
    (*t).sante = 100;
    int n =5; //nombre de stats
    //allocation dynamique du tableau
    (*t).stats = malloc(n * sizeof(double));
}
```

Vrai / Faux

- `malloc` initialise automatiquement la mémoire allouée à 0
- Allouer un tableau dynamique de 10 int : `malloc(10)`.
- La mémoire allouée avec `malloc` est stockée sur la pile.
- Une fois la mémoire libérée avec `free`, le pointeur devient automatiquement `NUL L`.
- Les variables locales déclarées dans une fonction sont stockées sur la pile.
- Le tas se libère automatiquement quand on sort d'une fonction.
- Si on perd l'adresse renvoyée par `malloc`, la mémoire devient impossible à libérer.
- Pour allouer une structure en dynamique, il faut écrire : `malloc(sizeof(struct))`.
- Une structure déclarée ainsi `Game g`; est stockée sur la pile.
- Les tableaux dynamiques obtenus via `malloc` fonctionnent comme des tableaux classiques.
- Une structure déclarée ainsi `Game *g = malloc(sizeof(Game))`; est stockée sur le tas.
- La pile se libère automatiquement quand on sort d'une fonction.
- Le pointeur qui pointe vers une zone du tas peut, lui, être placé sur la pile.
- Le tas est généralement plus petit que la pile.
- On peut faire un `malloc` à l'intérieur d'une fonction et utiliser la mémoire allouée dans le `main`.
- Si on oublie de faire `free`, la mémoire est automatiquement libérée quand on quitte la fonction.
- Il est correct d'écrire : `int *p; *p = 10;`.
- On peut faire `sizeof(ptr)` pour connaître la taille du bloc alloué par `malloc`.